# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

**COURSE NAME: COMPUTER NETWORKS**

**COURSE CODE: CSA07**

**COURSE OUTCOME COVERED**

*CO5: Measure network performance using key metrics with simulation tools. (BL4, BL5)*

Assessment Tool Weightage: 30%

### Annexure A — Pseudocode Writing Task (Algorithm Design)

### Code of Conduct

I (Y.Varun kumar reddy / 192424230) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Y.Varun kumar reddy

**SIMATS ENGINEERING**

**ASSESSMENT TOOL 2 — ANSWER SCRIPT**

Course: Computer Networks (CSA07)

*Course Outcome: CO5 — Measure network performance using key metrics with simulation tools (BL4, BL5)*

**Annexure A — Pseudocode Writing Task (Algorithm Design)**

# Question 1 — Network Throughput and Packet Loss Calculator

Problem: In Cisco Packet Tracer, total data received = 8000 bits in 4 seconds; packets sent = 200, packets received = 180. Design an algorithm to compute throughput (bps) and packet loss (%), validate inputs, classify the network, and display a full summary.

## Pseudocode

```
BEGIN NetworkPerformanceAnalysis

  // Step 1: Input the recorded simulation values
  INPUT total_bits_received      // e.g. 8000
  INPUT time_seconds             // e.g. 4
  INPUT packets_sent             // e.g. 200
  INPUT packets_received         // e.g. 180

  // Step 2: Validate inputs to avoid divide-by-zero errors
  IF (time_seconds == 0) OR (packets_sent == 0) THEN
      DISPLAY "Error: time and packets sent must not be zero"
      STOP
  END IF

  // Step 3: Calculate throughput in bits per second
  throughput = total_bits_received / time_seconds

  // Step 4: Calculate packet loss percentage
  lost_packets = packets_sent - packets_received
  packet_loss_percent = (lost_packets / packets_sent) * 100

  // Step 5: Classify network performance
  IF (throughput > 1500) AND (packet_loss_percent < 10) THEN
      status = "Efficient"
  ELSE
      status = "Inefficient"
  END IF

  // Step 6: Display complete performance summary
  DISPLAY "---- Network Performance Summary ----"
  DISPLAY "Throughput        : ", throughput, " bps"
  DISPLAY "Packet Loss       : ", packet_loss_percent, " %"
  DISPLAY "Network Status    : ", status

END NetworkPerformanceAnalysis
```

## Dry Run (with given values)

- throughput = 8000 / 4 = 2000 bps

- lost_packets = 200 − 180 = 20
- packet_loss_percent = (20 / 200) × 100 = 10 %
- Check: throughput (2000) > 1500 → true; packet_loss (10) < 10 → false ⇒ status = "Inefficient"

## Flowchart

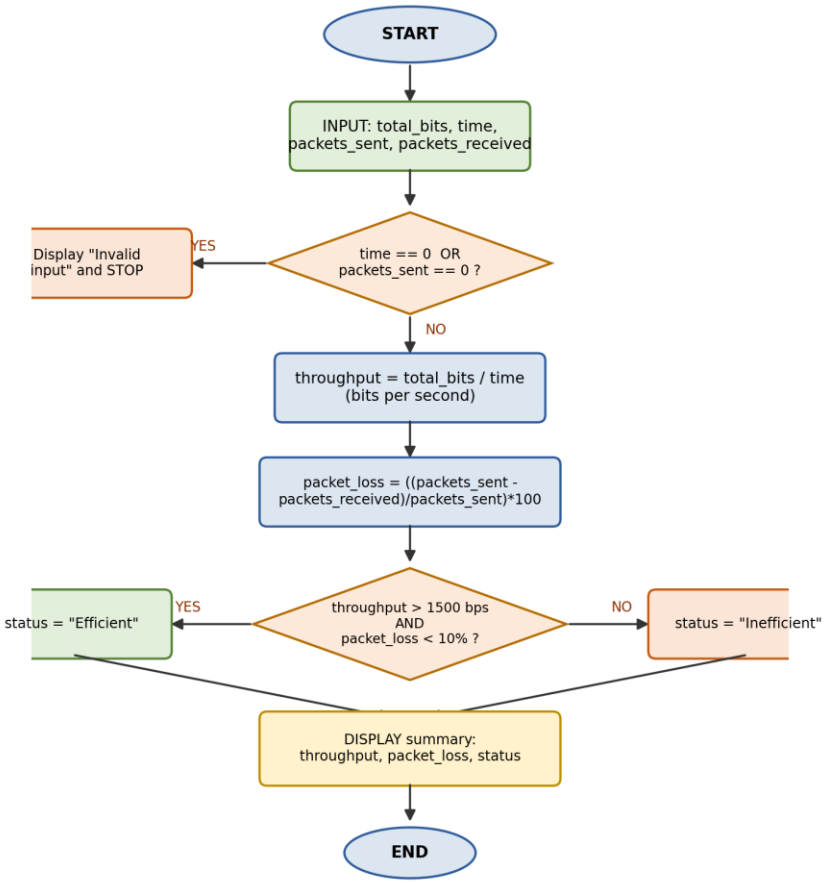


*Figure 1: Flow of throughput calculation, validation and classification*

## Explanation

The algorithm first validates that time and packets_sent are non-zero, preventing runtime division errors. Throughput is derived as total bits divided by elapsed time, giving a rate in bits per second, while packet loss percentage is derived from the difference between sent and received packets. A dual-condition check (throughput threshold AND packet loss threshold) then classifies the link as Efficient or Inefficient, and a single summary block reports all computed metrics together — matching how a network analyst would interpret Packet Tracer statistics.

# Question 2 — Bandwidth Utilization Monitoring and Overload Detection

Problem: An organization has multiple links connecting its branches. Design an algorithm that periodically collects bandwidth utilization of every link, stores the values, continuously monitors all links in a loop, and recommends switching to a backup link when utilization exceeds 80%.

## Pseudocode

```
BEGIN BandwidthMonitor

  // Step 1: Setup
  DECLARE link_list[1..N]        // list of N network links
  DECLARE backup_link[1..N]      // corresponding backup link for each link
  SET threshold = 80             // overload threshold in %

  // Step 2: Continuous monitoring loop
  REPEAT FOREVER
      FOR i = 1 TO N DO

          // Step 3: Collect current bandwidth utilization
          utilization[i] = GET_CURRENT_UTILIZATION(link_list[i])

          // Step 4: Check against threshold
          IF utilization[i] > threshold THEN
              DISPLAY "Warning: Link ", i, " overloaded at ", utilization[i]
              DISPLAY "Recommendation: switch to ", backup_link[i]
              SWITCH_TRAFFIC(link_list[i], backup_link[i])
          ELSE
              DISPLAY "Link ", i, " normal (", utilization[i], "%)"
          END IF

      END FOR

      WAIT for T seconds          // periodic sampling interval
  END REPEAT

END BandwidthMonitor
```

## Flowchart

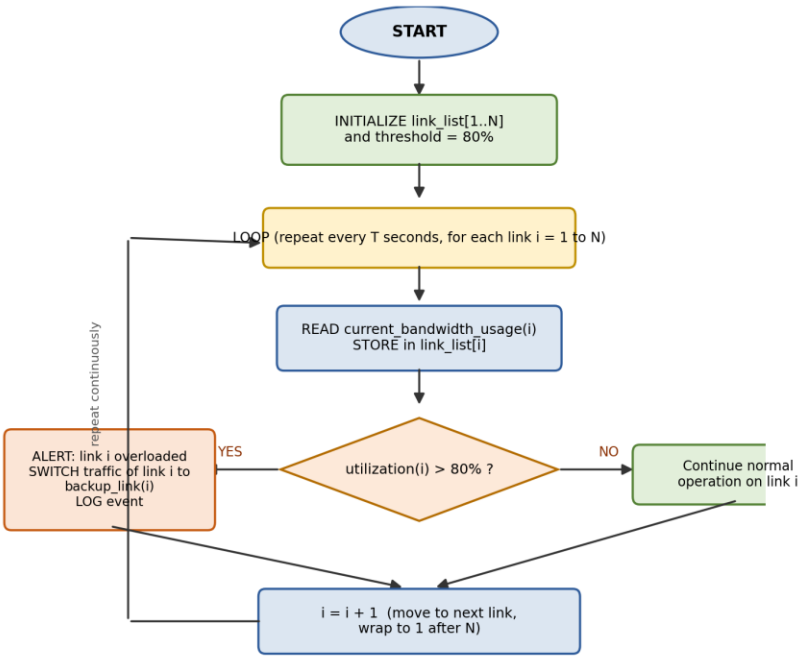


*Figure 2: Continuous per-link bandwidth monitoring and overload handling*

## How the Algorithm Supports Efficient Bandwidth Management

- Continuous monitoring: the infinite loop with a fixed sampling interval (T seconds) keeps utilization data current, so overload is detected quickly instead of being discovered after congestion occurs.
- Early warning: comparing utilization against a fixed 80% threshold flags a link before it saturates completely, rather than waiting for total failure.
- Automatic load redistribution: when a link is overloaded, traffic is proactively redirected to a pre-defined backup link, which balances load across the available paths.
- Congestion prevention: by acting at 80% rather than 100%, the algorithm leaves headroom, avoiding queuing delay, packet drops, and retransmissions caused by full saturation.
- Scalability: storing values in an array/list and looping over all links lets the same logic monitor any number of branch links without changing the algorithm structure.

# Question 3 — Best-Path Selection Using Link Quality Score (Multi-Branch Network)

Problem: Six branch offices connect to headquarters through multiple links, each with changing bandwidth, delay and utilization. Design an algorithm that periodically evaluates all links, computes a link-quality score, selects the best path, detects overloaded/failed links, switches to an alternate path, and alerts the administrator.

## Pseudocode

```
BEGIN BestPathSelection

  // Step 1: Setup — one array per metric, per branch link
  DECLARE bandwidth[1..N], delay[1..N], utilization[1..N]
  DECLARE score[1..N]
  SET w1, w2, w3 = weight constants for bandwidth, delay, utilization
  SET overload_threshold = 80

  // Step 2: Repeat monitoring at fixed intervals
  REPEAT FOREVER
      FOR i = 1 TO N DO

          // Step 3: Collect live metrics for each link
          bandwidth[i]   = GET_BANDWIDTH(link[i])
          delay[i]       = GET_DELAY(link[i])
          utilization[i] = GET_UTILIZATION(link[i])

          // Step 4: Detect overloaded or failed links
          IF (utilization[i] > overload_threshold)
               OR (LINK_STATUS(link[i]) == "DOWN") THEN
              MARK link[i] AS unavailable
              GENERATE_ALERT("Link ", i, " overloaded/failed")
              score[i] = -INFINITY   // remove from selection
          ELSE
              // Step 5: Compute weighted link-quality score
              score[i] = (w1 * bandwidth[i]) - (w2 * delay[i]) - (w3 * utilization[i])
          END IF

      END FOR

      // Step 6: Select the best available path
      best_path = INDEX of MAX(score[1..N])

      IF best_path was unavailable in previous cycle THEN
          GENERATE_ALERT("Rerouted to link ", best_path)
      END IF

      ROUTE_TRAFFIC(headquarters, branch, VIA best_path)
      DISPLAY "Selected path: ", best_path, " | Scores: ", score[1..N]

      WAIT for T seconds
  END REPEAT

END BestPathSelection
```

## Flowchart

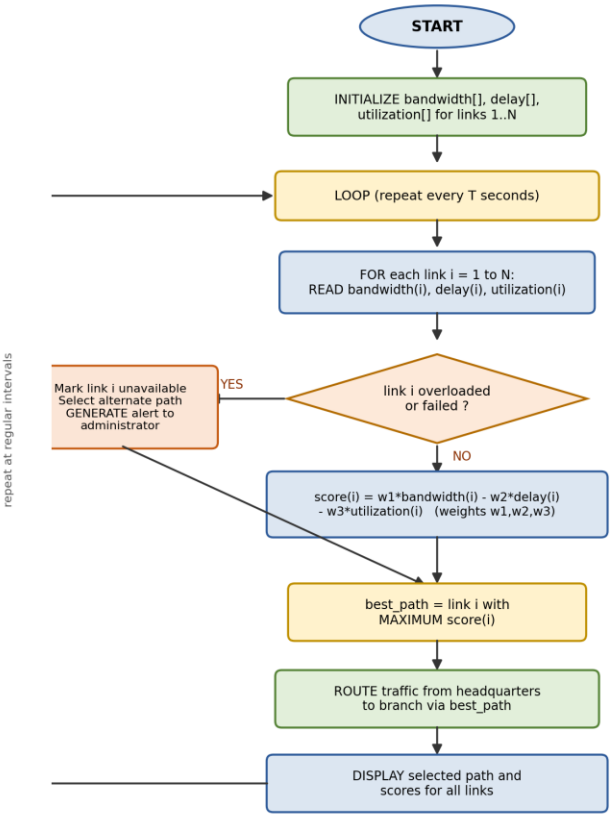


*Figure 3: Periodic link-quality scoring, best-path selection and failover*

## How the Algorithm Improves Reliability, Routing Efficiency and Fault Tolerance

- Network reliability: continuous re-evaluation at fixed intervals means the chosen path always reflects the link's current condition, not a one-time measurement, so the network self-corrects as conditions change.
- Routing efficiency: the composite score combines bandwidth, delay and utilization (instead of hop count or distance alone), so the algorithm picks the path that is actually fastest and least congested, not just the geographically shortest one.
- Fault tolerance: the overload/failure check removes a bad link from consideration immediately (score = −infinity) and forces selection of the next-best link, so a single link failure does not disrupt headquarters-to-branch communication.
- Administrator awareness: automatic alerts on overload, failure, or reroute events let the administrator take corrective action (e.g., provisioning more capacity) without having to poll link status manually.
- Adaptability: because the monitoring loop re-runs indefinitely, the algorithm adapts automatically to daily traffic pattern changes across all six branch links.